

AI Project 3

Name: Sarvansh Pachori

Class: TY C

Roll No.: 46

SRN No.: 202101087

Title: Board Game AI - Risk with Minimax-based AI

1) Problem Statement and Objectives:

Develop a graphical user interface to facilitate users for moves considering AI's decision-making by playing against it and observing its moves. Refine the implementation to enhance user experience, optimize AI performance, and ensure the game functions seamlessly.

Objectives:

1. **User-Friendly Interface:** Ensure the graphical user interface (GUI) is intuitive and easy to navigate for users of varying skill levels. Incorporate features such as tooltips, clear instructions, and intuitive controls to enhance user experience.
2. **Real-Time Feedback:** Provide real-time feedback to users during gameplay, such as highlighting recommended moves or displaying AI thought processes. This feature enhances user engagement and helps players understand the AI's decision-making rationale.
3. **Accessibility Features:** Consider incorporating accessibility features such as adjustable font sizes, color-blind mode, and keyboard shortcuts to make the game accessible to a wider audience, including users with disabilities.
4. **Testing and Quality Assurance:** Conduct thorough testing of the GUI and AI functionalities to identify and rectify any bugs, glitches, or usability issues. Utilize techniques such as unit testing, integration testing, and user acceptance testing to ensure the final product meets quality standards and delivers a seamless gaming experience.

2) Methodology details:

Implement Algorithm:

- a) **Develop the necessary data structures** to represent the game state, player territories, and troop placements.
- b) **Design of GUI Layout:** Design a graphical user interface (GUI) layout that effectively represents the game board, territories, player actions, and game state. Use libraries like Tkinter to create interactive buttons, labels, and canvas for displaying the game elements.
- c) **Implementation of Game Logic:** Implement classes for territories, continents, players, and AI players. Develop functions/methods for actions such as claiming territories, placing troops, moving troops, and attacking territories. Ensure that the game logic accurately reflects the rules of "Risk" and enables interaction between players and the AI.
- d) **User Experience Enhancement:** Enhance the GUI design to improve user experience and aesthetics. Implement features such as tooltips, status indicators, and animations to provide visual feedback to players. Optimize interface responsiveness and usability for smooth gameplay
- e) **Implement the Minimax algorithm** with alpha-beta pruning to explore the solution space efficiently. This algorithm will help the AI to make optimal decisions by considering different possible moves, opponent's responses, and their outcomes.
- f) **Utilize backtracking** to navigate through the game tree. If a promising move is found, the algorithm will explore it further; if not, it will backtrack and explore other moves.

Validation and Testing:

- a) **Unit Testing:** Perform unit testing on individual functions to ensure they work as expected. This includes testing functions such as create_territory_buttons, update_territory_buttons, switch_player, claim_territory, place_troops, move_troops, attack_territory, resolve_attack, roll_dice, compare_dice_rolls, and gameWindow. For each function, test different scenarios, including valid inputs, edge cases, and invalid inputs to ensure robustness and correctness
- b) **Integration Testing:** Conduct integration testing to verify that different components of the game interact correctly with each other.

Test the interaction between UI components (buttons, labels) and game logic functions to ensure seamless gameplay

- c) **User Interface Testing:** Test the user interface components, such as buttons, labels, and windows, to ensure they are displayed correctly and respond appropriately to user interactions.

Verify that the game window, map image, and player information are displayed correctly.

Ensure that buttons for placing troops, moving troops, and attacking are functional and responsive

- d) **Test the AI** with various game scenarios, starting from simpler ones and gradually increasing to more complex scenarios. This could involve games with different numbers of players, different initial troop placements, etc.

- e) **Validate the correctness of AI's decisions** against known strategies for the Risk game. For example, you could compare the AI's moves with the moves a human expert would make in the same situation.

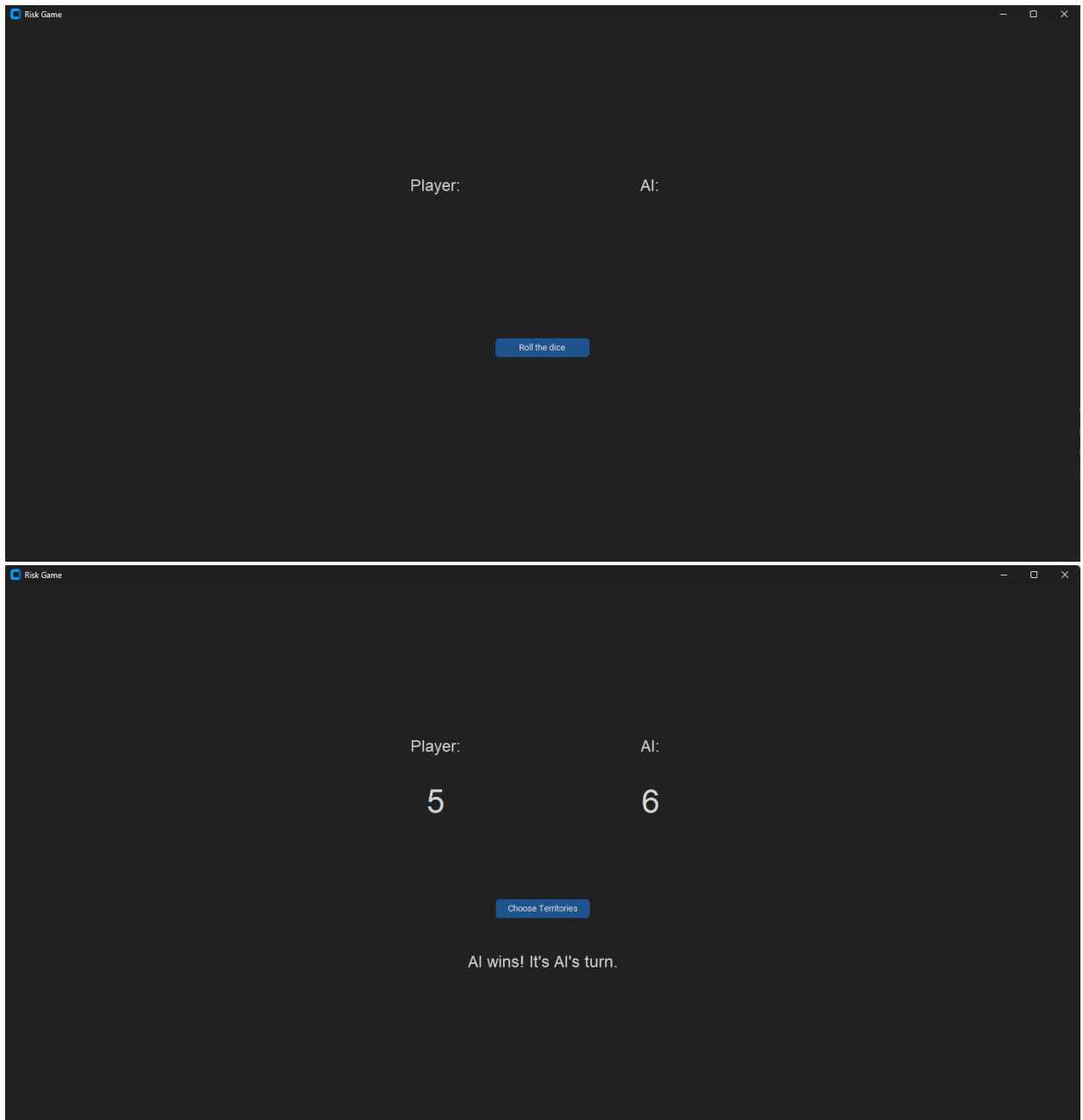
- f) **Measure the AI's performance** in terms of execution time and memory usage. The goal to ensure that the AI can make decisions within a reasonable time frame, even in complex scenarios with a large number of possible moves.

3) **Game Rules:**

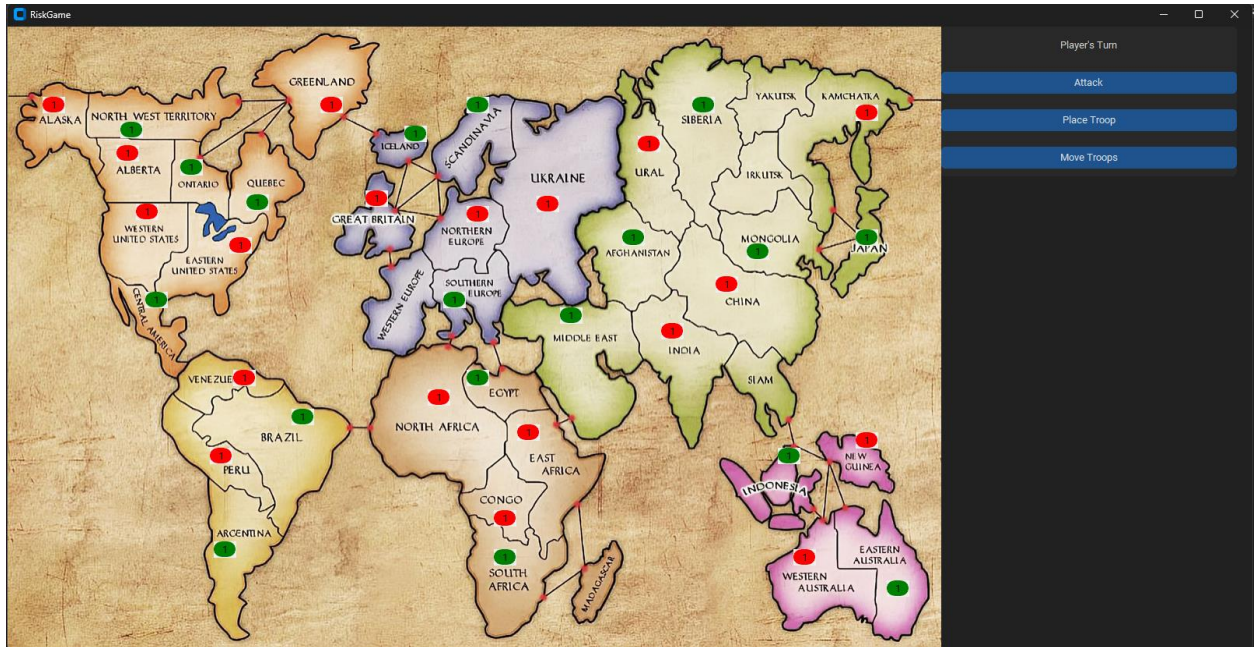
- **Objective:** Control all territories on the map to win.
- **Turn-based Gameplay:** Players take turns placing troops, attacking enemy territories, and moving troops.
- **Initial Troop Placement:** Players start by placing troops on their territories.
- **Game Phases:**
 - **Place Troops:** At the start of each turn, players can add troops to their territories.
 - **Attack Phase:** Players can attack adjacent enemy territories.
 - **Move Troops:** After attacking, players can move troops between their territories.
- **Attacking:** Roll dice to determine attack outcomes.

- **Winning the Game:** The game ends when one player controls all territories.

4) GUI Screenshots:



first all the territories needs to be claimed. At the starting all the territory buttons are grey showing these territories have no owner if **Player** owns a territory its button becomes **green** and number of armies in the territories are displayed inside the button and for **AI** the button color is **red**.



After all the territories have been claimed the player or ai chooses to attack/place troops/move troops, actions performed by ai are shown below the buttons.

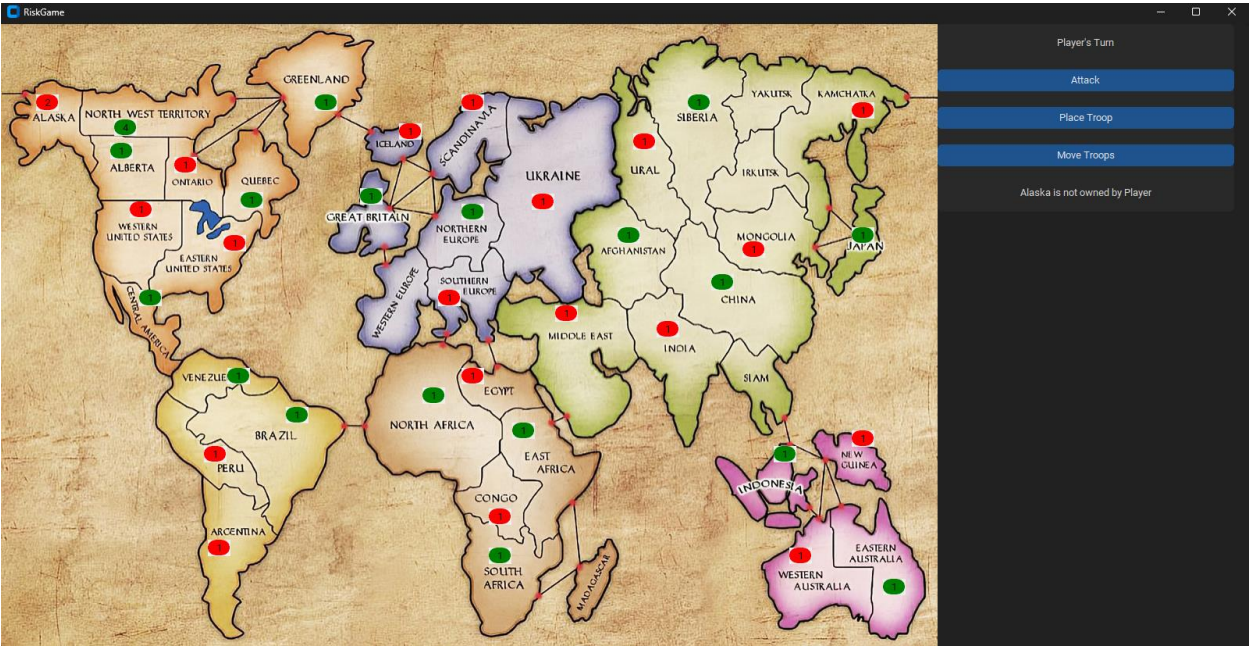
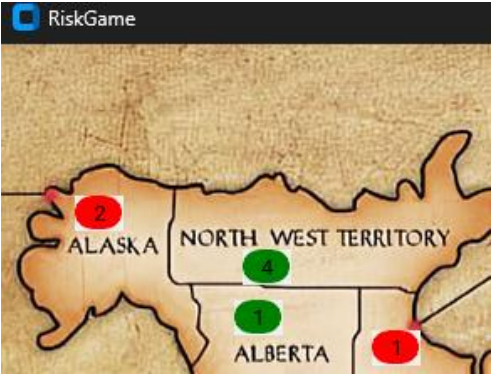
Here player chose to put 3 troops N .W territory



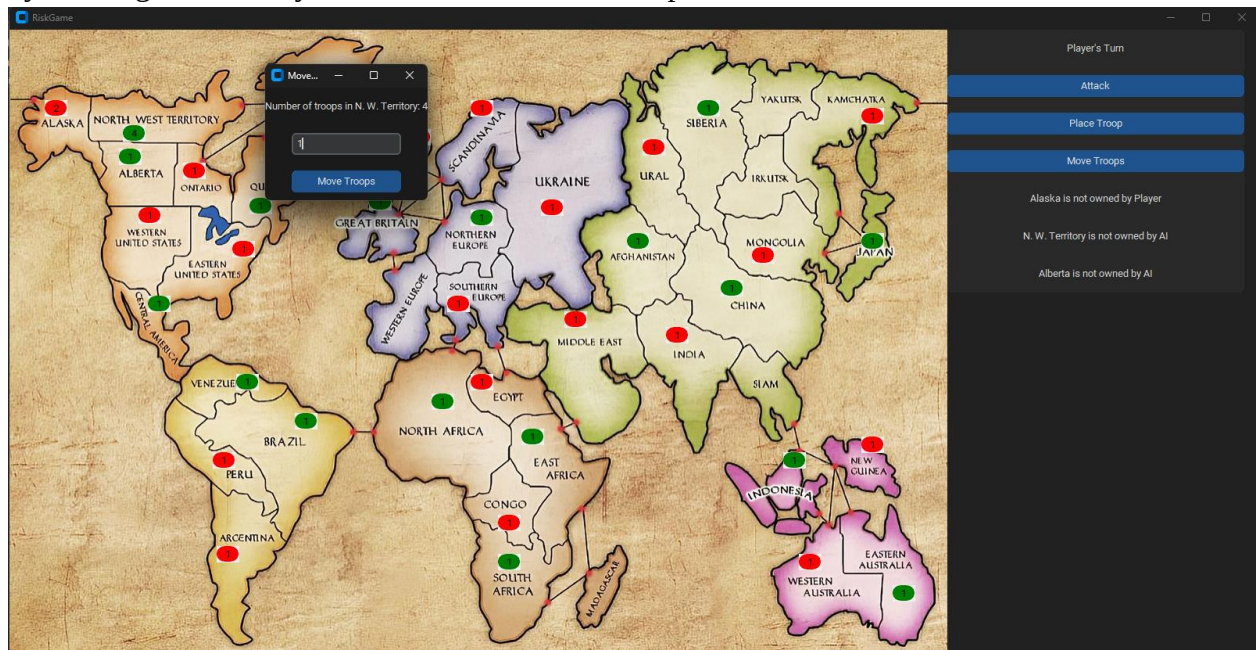
After the troops are placed number of troops in the territory is updated inside the button.3 troops were added to the territory making it 4 in total.



AI put 1 troop in the Alaska



When “move troops” button is clicked player is asked to select “from” and “to” territories by clicking the territory buttons and number of troops to move is entered .



After the troops are moved army numbers are updated



When “Attack” button is clicked same from and to territories are asked and number of troops to use are asked it should be 1 less than total number of troops in the territory.



Player attacked from N.W. territory to Alaska with 2 troops then dices are rolled and player wins so Alaska is now Player's territory so owner is updated and territory button becomes green.



Now AI tried to attack from Ontario to Alberta but lost the attack by dice comparison so the troops used by AI was 2 so they are destroyed so remaining troops in Ontario becomes 1.

5) **Expected Output:**

Observation:

Upon reviewing the Risk game project, it is evident that the implementation closely aligns with the game rules as described in the previous modules. The fundamental principles of Risk, including player turns, troop placement, troop movement, and combat resolution, are faithfully represented in the game mechanics.

One notable aspect is the adherence to the fixed number of troops allocated to each player at the beginning of the game, as outlined in the previous modules. Players are provided with a predetermined number of troops during the initial troop placement phase, reflecting the strategic aspect of resource allocation at the start of the game. This adherence to the specified game rule ensures a consistent and balanced gameplay experience for all participants.

Furthermore, the implementation demonstrates an understanding of the territorial control mechanics inherent to Risk. Players are required to strategically claim territories, reinforce their positions, and engage in tactical combat to expand their dominance across the game board. The mechanics governing troop movement, territory ownership, and continent bonuses effectively capture the essence of territorial conquest central to the Risk game experience.

Overall, the inclusion of the Minimax algorithm enriches the project by introducing a sophisticated AI opponent that challenges players to think strategically and make informed decisions to outmaneuver their virtual adversary. This aspect contributes to the project's success in delivering an engaging and immersive Risk gaming experience that remains true to the strategic depth and complexity of the original game.

Conclusion:

The Risk game project successfully implements the core mechanics and strategic elements of the classic board game while introducing modern enhancements such as a graphical user interface and an AI opponent powered by the Minimax algorithm. Through the utilization of Python and libraries such as Tkinter and PIL, the project delivers an immersive gaming experience that remains faithful to the original rules and objectives of Risk.

In conclusion, the Risk game project effectively combines classic gameplay mechanics with modern programming techniques to deliver an enjoyable and immersive gaming experience. With its faithful representation of the original game rules, intuitive user interface, and challenging AI opponent, the project stands as a testament to the enduring appeal of Risk and showcases the potential for innovation within the realm of digital board games.

- 6) **Source code:** In all the screenshots above AI-Player is playing its own moves based on results from minmax algorithm

This is the AIPlayer class where minmax algorithm function is present making the decision

```
from Player import Player
import time
import random

class AIPlayer(Player):
    def __init__(self, name):
        super().__init__(name)

    def minimax(self, game, depth, is_maximizing, player2):
        if depth == 0 or game.is_over():
            return self.evaluate(game)

        if is_maximizing:
            best_score = float('-inf')
            for move in game.get_possible_moves(self):
                game.apply_move(move)
                score = self.minimax(game, depth - 1, False, player2)
                game.undo_move(move)
                if move['action'] == 'attack':
                    score += 30 # Add a large bonus for 'attack' moves
                elif move['action'] == 'move' and any(adjacent_territory.owner !=
self for adjacent_territory in
                                                    move['from_territory'].adja
cent_territories):
                    score += 20 # Add a smaller bonus for 'move' actions that
position the player to attack
                elif move['action'] == 'place' :
                    score += 10 # Add a small bonus for 'place' actions that
reinforce the player's territories
                best_score = max(score, best_score)
            return best_score
        else:
```



```

        best_score = float('inf')
        for move in game.get_possible_moves(player2):
            game.apply_move(move)
            score = self.minimax(game, depth - 1, True, player2)
            game.undo_move(move)
            best_score = min(score, best_score)
        return best_score

def evaluate(self, game):
    score = 0
    # Increase score for each territory owned by the AI
    score += len(self.territories)
    # Increase score for each continent owned by the AI
    score += len(self.continents) * 5 # Assume owning a continent is 5 times
as valuable as owning a territory
    # Increase score for each army owned by the AI
    score += self.armies
    return score

def get_num_armies(game, move, action):
    # This is a simple example that always returns 1.
    # You should replace this with your own logic.
    return 1

def choose_and_apply_move(self, game, player2):
    # Use the minimax function to choose the best move
    best_score = float('-inf')
    best_move = None
    possible_moves = game.get_possible_moves(self)
    for move in possible_moves:
        # Make the move
        game.apply_move(move)
        # Use the minimax function to get the score of the current game state
        score = self.minimax(game, 3, False, player2) # Adjust the depth as
needed
        # Undo the move
        game.undo_move(move)
        # Update the best score and best move
        if score > best_score:
            best_score = score
            best_move = move

    # After all moves have been evaluated, apply the best move
    game.apply_move(best_move)
    # If the action is 'attack', roll the dice and perform the attack

```

```

if best_move['action'] == 'attack':
    for continent in game.continents:
        for territory in continent.territories:
            if territory.name == best_move['from_territory']:
                from_territory = territory
            if territory.name == best_move['to_territory']:
                to_territory = territory
        num_armies = best_move['num_armies']

        attack_dice = sorted([random.randint(1, 6) for _ in
range(num_armies)], reverse=True)
        defense_dice = sorted([random.randint(1, 6) for _ in
range(min(to_territory.armies, 2))], reverse=True)

        # Print the dice rolls
        print(f"{self.name} rolled {attack_dice} for the attack")
        print(f"{to_territory.owner.name} rolled {defense_dice} for the
defense")

        # Compare dice and remove armies
        for attack_die, defense_die in zip(attack_dice, defense_dice):
            if attack_die > defense_die:
                to_territory.armies -= 2
                if to_territory.armies < 0:
                    to_territory.armies = 0
                if to_territory.armies == 0:
                    # Remove the territory from the territories list of the
previous owner
                    to_territory.owner.territories.remove(to_territory)
                    # Change the owner of the territory and add it to the
territories list of the new owner
                    to_territory.owner = self
                    self.territories.append(to_territory)
                    to_territory.armies = num_armies
            else:
                self.armies += num_armies
        # Decrease the number of armies in from_territory by num_armies
        from_territory.armies -= num_armies

        if to_territory.owner == self:
            print("The attack was successful!")
        else:
            print("The attack was not successful.")

        # Display current player's owned territories with their army numbers

```

```

        print(f"{self.name}'s territories after the attack:")
        for i, territory in enumerate(self.territories):
            print(f"{i + 1}. {territory.name} (Armies: {territory.armies})")

    return best_move

def update_num_armies(self, num_armies):
    # Update the number of armies the AI player has
    self.num_armies = num_armies

```

Below are the Player, Continents and Territories classes:

```

class Continent:
    def __init__(self, name, bonus):
        self.name = name
        self.bonus = bonus
        self.territories = []

    def add_territory(self, territory):
        self.territories.append(territory)

    def is_controlled_by(self, player):
        return all(territory.owner == player for territory in self.territories)

```

```

class Player:
    def __init__(self, name):
        self.name = name
        self.territories = []
        self.continents = []
        self.armies = 35

    def add_territory(self, territory):
        self.territories.append(territory)
        territory.owner = self

    def add_continent(self, continent):
        self.continents.append(continent)

    def add_armies(self, num):
        self.armies += num

    def del_armies(self):
        self.armies -= 1

```

```

class Territory:
    def __init__(self, name, continent):
        self.name = name

```

```

        self.continent = continent
        self.owner = None
        self.armies = 0
        self.adjacent_territories = []

    def add_adjacent_territory(self, territory):
        self.adjacent_territories.append(territory)

    def set_owner(self, player):
        self.owner = player
        self.armies = 1 # Set the initial number of armies to 1 when a territory
is claimed

    def update_armies(self, num_armies):
        self.armies = num_armies

```

Below is the main.py which initializes the starting GUI window where dices are rolled to see who goes first

```

import customtkinter as ctk
from PIL import Image, ImageTk
import random
import time
import mainGame

def close_current_window():
    app.destroy() # Close the current window

def navigate_to_new_page(winner):
    close_current_window() # Close the current window
    mainGame.gameWindow(winner) # Call the gameWindow function with the winner
parameter

def choose_territories():
    navigate_to_new_page(winner) # Navigate to the gameWindow with the winner
set to "Player"

def roll_dice():
    # Disable the button during dice rolling animation
    roll_button.configure(state=ctk.DISABLED)

    # Display rolling animation for 2 seconds
    for _ in range(20):
        dice_value_player.configure(text=str(random.randint(1, 6)))
        dice_value_ai.configure(text=str(random.randint(1, 6)))
        app.update()

```



```

        time.sleep(0.1)

    # Enable the button after dice rolling animation
    roll_button.configure(state=ctk.NORMAL)

    # Get the dice values
    player_dice = int(dice_value_player.cget("text"))
    ai_dice = int(dice_value_ai.cget("text"))

    # Determine the winner
    global winner
    if player_dice > ai_dice:
        winner_label.configure(text="Player wins! It's their turn.")
        winner = "Player"
        # Change button text and command
        roll_button.configure(text="Choose Territories",
command=choose_territories)
    elif player_dice < ai_dice:
        winner_label.configure(text="AI wins! It's AI's turn.")
        winner = "AI"
        # Change button text and command
        roll_button.configure(text="Choose Territories",
command=choose_territories)
    else:
        winner_label.configure(text="It's a tie! Roll again.")
        winner = None # Set winner to None in case of tie

        # Change button text and command
        roll_button.configure(text="Roll the dice", command=roll_dice)

app = ctk.CTk()
app.title("Risk Game")
ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("dark-blue")
app.geometry("1600x800")

frame = ctk.CTkFrame(master=app, width=1600, height=800)
frame.pack(pady=0, padx=0, fill="both", expand=True)

# Create Labels to indicate players
player_label = ctk.CTkLabel(frame, text="Player:", font=("Helvetica", 24))
player_label.place(relx=0.4, rely=0.3, anchor="center")

ai_label = ctk.CTkLabel(frame, text="AI:", font=("Helvetica", 24))
ai_label.place(relx=0.6, rely=0.3, anchor="center")

```

```

# Create widgets
dice_value_player = ctk.CTkLabel(frame, text="", font=("Helvetica", 48))
dice_value_player.place(relx=0.4, rely=0.4, anchor="center")

dice_value_ai = ctk.CTkLabel(frame, text="", font=("Helvetica", 48))
dice_value_ai.place(relx=0.6, rely=0.4, anchor="center")

roll_button = ctk.CTkButton(frame, text="Roll the dice", command=roll_dice)
roll_button.place(relx=0.5, rely=0.6, anchor="center")

winner_label = ctk.CTkLabel(frame, text="", font=("Helvetica", 24))
winner_label.place(relx=0.5, rely=0.7, anchor="center")

app.mainloop()

```

Below is the main game window where the Risk Game map all the buttons etc are initialized and functions for all of it

```

import customtkinter as ctk
from PIL import Image, ImageTk
from Territory import Territory
from Continents import Continent
from Player import Player
from AIPlayer import AIPlayer
# Create continents
north_america = Continent("North America", 5)
south_america = Continent("South America", 2)
europe = Continent("Europe", 5)
africa = Continent("Africa", 3)
asia = Continent("Asia", 7)
australia = Continent("Australia", 2)

# Create territories
alaska = Territory("Alaska", north_america)

# Add territories to their respective continents
north_america.add_territory(alaska)

player = Player("Player")
ai = AIPlayer("AI")

# Create territories
territories2 = {

```

```

"Alaska": Territory("Alaska", north_america),
"N. W. Territory": Territory("N. W. Territory", north_america),
"Alberta": Territory("Alberta", north_america),
"Ontario": Territory("Ontario", north_america),
"Quebec": Territory("Quebec", north_america),
"Greenland": Territory("Greenland", north_america),
"W. U.S.": Territory("W. U.S.", north_america),
"E. U.S.": Territory("E. U.S.", north_america),
"C. America": Territory("C. America", north_america),
"Venezuela": Territory("Venezuela", south_america),
"Brazil": Territory("Brazil", south_america),
"Peru": Territory("Peru", south_america),
"Argentina": Territory("Argentina", south_america),
"Iceland": Territory("Iceland", europe),
"G. Britain": Territory("G. Britain", europe),
"N. Europe": Territory("N. Europe", europe),
"Scandinavia": Territory("Scandinavia", europe),
"Ukraine": Territory("Ukraine", europe),
"S. Europe": Territory("S. Europe", europe),
"N. Africa": Territory("N. Africa", africa),
"Egypt": Territory("Egypt", africa),
"E. Africa": Territory("E. Africa", africa),
"Congo": Territory("Congo", africa),
"S. Africa": Territory("S. Africa", africa),
"Siberia": Territory("Siberia", asia),
"Ural": Territory("Ural", asia),
"Afghanistan": Territory("Afghanistan", asia),
"M. East": Territory("M. East", asia),
"India": Territory("India", asia),
"China": Territory("China", asia),
"Mongolia": Territory("Mongolia", asia),
"Japan": Territory("Japan", asia),
"Kamchatka": Territory("Kamchatka", asia),
"Indonesia": Territory("Indonesia", australia),
"W. Australia": Territory("W. Australia", australia),
"E. Australia": Territory("E. Australia", australia),
"New Guinea": Territory("New Guinea", australia)
}

# Add adjacent territories for each territory object
territories2["Alaska"].add_adjacent_territory(territories2["N. W. Territory"])
territories2["Alaska"].add_adjacent_territory(territories2["Alberta"])
# ... (continue for other territories)

# Example for one territory

```

```

territories2["N. W. Territory"].add_adjacent_territory(territories2["Alaska"])
territories2["N. W. Territory"].add_adjacent_territory(territories2["Alberta"])
territories2["N. W. Territory"].add_adjacent_territory(territories2["Ontario"])
territories2["N. W. Territory"].add_adjacent_territory(territories2["Greenland"])
# ... (continue for other territories)

```

Note: The above code is an example. You'll need to add the actual adjacent territories based on the game rules and the image provided.

```

def create_territory_buttons(canvas,current_player):
    global territories2, player, ai
    # Define the coordinates and shapes of the territories
    territories = {
        "Alaska": [(10, 50), (100, 50), (100, 150), (10, 150)],
        "N. W. Territory": [(70, 70), (240, 70), (240, 195), (70, 195)],
        "Alberta": [(100, 100), (200, 100), (200, 225), (100, 225)],
        "Ontario": [(170, 80), (295, 80), (295, 280), (170, 280)],
        "Quebec": [(255, 150), (380, 150), (380, 300), (255, 300)],
        "Greenland": [(350, 50), (475, 50), (475, 150), (350, 150)],
        "W. U.S.": [(100, 175), (250, 175), (250, 300), (100, 300)],
        "E. U.S.": [(220, 230), (370, 230), (370, 330), (220, 330)],
        "C. America": [(150, 300), (225, 300), (225, 400), (150, 400)],
        "Venezuela": [(250, 400), (350, 400), (350, 500), (250, 500)], #button
nhi milla
        "Brazil": [(300, 450), (450, 450), (450, 550), (300, 550)],
        "Peru": [(220, 500), (320, 500), (320, 600), (220, 600)],
        "Argentina": [(200, 620), (350, 620), (350, 720), (200, 720)],
        "Iceland": [(470, 100), (570, 100), (570, 175), (470, 175)],
        "G. Britain": [(420, 170), (520, 170), (520, 270), (420, 270)],
        "N. Europe": [(550, 190), (650, 190), (650, 290), (550, 290)],
        "Scandinavia": [(550, 50), (650, 50), (650, 150), (550, 150)],
        "Ukraine": [(640, 190), (740, 190), (740, 265), (640, 265)],
        "S. Europe": [(520, 300), (620, 300), (620, 400), (520, 400)],
        "N. Africa": [(500, 450), (500, 450), (600, 500), (600, 500)],
        "Egypt": [(550, 400), (650, 400), (650, 500), (550, 500)],
        "E. Africa": [(650, 470), (680, 470), (680, 570), (650, 570)],
        "Congo": [(620, 580), (650, 580), (650, 680), (620, 680)],
        "S. Africa": [(620, 630), (650, 630), (650, 730), (620, 730)],
        "Siberia": [(840, 100), (940, 100), (940, 100), (840, 100)],
        "Ural": [(770, 150), (870, 150), (870, 150), (770, 150)],
        "Afghanistan": [(750, 270), (850, 270), (850, 270), (750, 270)],
        "M. East": [(670, 370), (770, 370), (770, 370), (670, 370)],
        "India": [(800, 390), (900, 390), (900, 390), (800, 390)],
        "China": [(870, 330), (970, 330), (970, 330), (870, 330)],
    }

```



```

    "Mongolia": [(910, 288), (1010, 288), (1010, 288), (910, 288)],
    "Japan": [(1050, 270), (1150, 270), (1150, 270), (1050, 270)],
    "Kamchatka": [(1050, 110), (1150, 110), (1150, 110), (1050, 110)],
    "Indonesia": [(950, 500), (1050, 500), (1050, 600), (950, 600)],
    "W. Australia": [(970, 630), (1070, 630), (1070, 730), (970, 730)],
    "E. Australia": [(1090, 670), (1190, 670), (1190, 770), (1090, 770)],
    "New Guinea": [(1050, 480), (1150, 480), (1150, 580), (1050, 580)]
}

# Create the territory buttons
territory_buttons = {}
for name, coords in territories.items():
    x1, y1 = coords[0]
    x2, y2 = coords[2]
    button_x = (x1 + x2) // 2
    button_y = (y1 + y2) // 2
    territory = territories2[name]
    button_color = "gray"
    if territory.owner is not None:
        button_color = "green" if territory.owner == current_player else
"red"

    # Update existing button or create a new one if it doesn't exist
    if name in territory_buttons:
        button = territory_buttons[name]
        button.configure(fg_color=button_color, text=str(territory.armies))
    else:
        button = ctk.CTkButton(canvas, text=str(territory.armies), width=20,
height=20, corner_radius=10,
fg_color=button_color, hover_color="light gray",
text_color="black")
        button.place(x=button_x - 10, y=button_y - 10)
        button.bind("<Button-1>", lambda event, territory=territory:
claim_territory(territory, territory_buttons))
        button.territory = territory # Store the territory object in the button
        territory_buttons[name] = button

return territory_buttons

def update_territory_buttons(territory_buttons, territory):
    button = territory_buttons[territory.name]
    button.configure(text=str(territory.armies),
fg_color="gray" if territory.owner is None else ("green" if
territory.owner == player else "red"))

```

```

def switch_player():
    global current_player, player, ai, territories2

    # Check if all territories are owned by the same player
    all_territories_owned = all(territory.owner == current_player for territory
in territories2.values())

    if all_territories_owned:
        # Display a small window announcing the winner
        winner_window = ctk.CTkToplevel()
        winner_window.title("Game Over")
        winner_window.geometry("300x100")
        winner_label = ctk.CTkLabel(winner_window, text=f"Winner:
{current_player.name}")
        winner_label.pack(pady=20)
        return

    # Switch to the next player's turn
    if current_player == player:
        current_player = ai
        current_player_label.configure(text=f"{ai.name}'s Turn")
    else:
        current_player = player
        current_player_label.configure(text=f"{player.name}'s Turn")

def claim_territory(territory, territory_buttons):
    if territory.owner is None:
        territory.set_owner(current_player)
        current_player.add_territory(territory)
        update_territory_buttons(territory_buttons, territory)
        switch_player()
    else:
        print(f"{territory.name} is already owned by {territory.owner.name}")

# Create continents
north_america = Continent("North America", 5)
europe = Continent("Europe", 5)
asia = Continent("Asia", 7)
africa = Continent("Africa", 3)
australia = Continent("Australia", 2)
south_america = Continent("South America", 2)

```

```

# # Add adjacent territories
# # Note: You'll need to add the actual adjacent territories based on the game
rules
# for territory in territories.values():
#     territory.add_adjacent_territory(territory)

# # Add territories to continents
# for territory in territories.values():
#     territory.continent.add_territory(territory)
def place_troops(territory_buttons, small_section, root, current_player):

    # Display a label asking the player to choose a territory
    choose_territory_label = ctk.CTkLabel(small_section, text="Choose a territory
to place troops")
    choose_territory_label.pack(pady=10)

    # Define a function to handle territory selection
    def select_territory(territory):
        nonlocal choose_territory_label

        # Check if the territory is owned by the current player
        # Check if the territory is owned by the current player
        if territory.owner is not None and territory.owner.name ==
current_player.name:
            choose_territory_label.pack_forget() # Remove the "Choose a
territory" label

            # Create a new window to prompt for the number of troops
            troops_window = ctk.CTkToplevel(root)
            troops_window.attributes("-topmost", True)
            troops_window.title("Place Troops")

            # Create a label and entry field for the number of troops
            troops_label = ctk.CTkLabel(troops_window, text="Enter the number of
troops (1-3):")
            troops_label.pack(pady=10)

            troops_entry = ctk.CTkEntry(troops_window)
            troops_entry.pack(pady=10)

            # Define a function to handle troop placement
            def place_troops_callback():
                nonlocal troops_window

                # Get the number of troops from the entry field

```

```

        troops_count = troops_entry.get()

        # Validate the input
        try:
            troops_count = int(troops_count)
            if 1 <= troops_count <= 3:
                # Add the troops to the territory
                territory.armies += troops_count

                # Update the territory button
                update_territory_buttons(territory_buttons, territory)

                # Close the troops window
                troops_window.destroy()

                # Switch to the next player's turn
                switch_player()
            else:
                # Display an error message if the input is invalid
                ctk.CTkLabel(troops_window, text="Invalid input. Please
enter a number between 1 and 3.").pack(pady=10)
        except ValueError:
            # Display an error message if the input is not a number
            ctk.CTkLabel(troops_window, text="Invalid input. Please enter
a number.").pack(pady=10)

        # Create a button to confirm troop placement
        confirm_button = ctk.CTkButton(troops_window, text="Place Troops",
command=place_troops_callback)
        confirm_button.pack(pady=10)

    else:
        # Display an error message if the territory is not owned by the
current player
        ctk.CTkLabel(small_section, text=f"{territory.name} is not owned by
{current_player.name}").pack(pady=10)

    # Bind the select_territory function to each territory button
    def handle_button_click(event, territory):
        select_territory(territory)

    # Bind the handle_button_click function to each territory button
    for button in territory_buttons.values():
        button.bind("<Button-1>", lambda event, territory=button.territory:
handle_button_click(event, territory))

```

```

def move_troops(territory_buttons, small_section, root, current_player):
    global territories2

    # Display a Label asking the player to choose a territory to move troops from
    choose_territory_label = ctk.CTkLabel(small_section, text="Choose a territory
to move troops from")
    choose_territory_label.pack(pady=10)

    def select_from_territory(territory):
        nonlocal choose_territory_label

        # Check if the territory is owned by the current player
        if territory.owner is not None and territory.owner.name ==
current_player.name:
            choose_territory_label.pack_forget() # Remove the "Choose a
territory" Label

            # Display a Label asking the player to choose a territory to move
troops to
            choose_territory_label = ctk.CTkLabel(small_section, text="Choose a
territory to move troops to")
            choose_territory_label.pack(pady=10)

            def select_to_territory(to_territory):
                nonlocal choose_territory_label

                # Check if the territories are adjacent and owned by the same
player
                if (
                    to_territory in territory.adjacent_territories
                    and to_territory.owner is not None
                    and to_territory.owner.name == current_player.name
                ):
                    choose_territory_label.pack_forget() # Remove the "Choose a
territory" Label

                    # Create a new window to prompt for the number of troops to
move
                    troops_window = ctk.CTkToplevel(root)
                    troops_window.title("Move Troops")
                    troops_window.attributes("-topmost", True) # Make the window
appear above the root window

```



```

        # Create a label displaying the number of troops in the
        "from" territory
        troops_label = ctk.CTkLabel(troops_window, text=f"Number of
troops in {territory.name}: {territory.armies}")
        troops_label.pack(pady=10)

        # Create an entry field for the number of troops to move
        troops_entry = ctk.CTkEntry(troops_window)
        troops_entry.pack(pady=10)

        # Define a function to handle troop movement
        def move_troops_callback():
            nonlocal troops_window

            # Get the number of troops to move from the entry field
            troops_count = troops_entry.get()

            # Validate the input
            try:
                troops_count = int(troops_count)
                if 1 <= troops_count < territory.armies:
                    # Subtract the troops from the "from" territory
                    territory.armies -= troops_count

                    # Add the troops to the "to" territory
                    to_territory.armies += troops_count

                    # Update the territory buttons
                    update_territory_buttons(territory_buttons,
territory)

                    update_territory_buttons(territory_buttons,
to_territory)

                    # Close the troops window
                    troops_window.destroy()

                    # Switch to the next player's turn
                    switch_player()
            except:
                # Display an error message if the input is
invalid
                ctk.CTkLabel(troops_window, text="Invalid input.
Please enter a number between 1 and one less than the total
troops.").pack(pady=10)
            except ValueError:

```

```

        # Display an error message if the input is not a
number
        ctk.CTkLabel(troops_window, text="Invalid input.
Please enter a number.").pack(pady=10)

        # Create a button to confirm troop movement
        confirm_button = ctk.CTkButton(troops_window, text="Move
Troops", command=move_troops_callback)
        confirm_button.pack(pady=10)

    else:
        # Display an error message if the territories are not
adjacent or not owned by the same player
        if to_territory not in territory.adjacent_territories:
            ctk.CTkLabel(small_section, text=f"{to_territory.name} is
not adjacent to {territory.name}").pack(pady=10)
            elif to_territory.owner is None or to_territory.owner.name !=
current_player.name:
                ctk.CTkLabel(small_section, text=f"{to_territory.name} is
not owned by {current_player.name}").pack(pady=10)

        # Bind the select_to_territory function to each territory button
        for button in territory_buttons.values():
            button.bind("<Button-1>", lambda event,
territory=button.territory: select_to_territory(territory))

    else:
        # Display an error message if the territory is not owned by the
current player
        ctk.CTkLabel(small_section, text=f"{territory.name} is not owned by
{current_player.name}").pack(pady=10)

        # Bind the select_from_territory function to each territory button
        for button in territory_buttons.values():
            button.bind("<Button-1>", lambda event, territory=button.territory:
select_from_territory(territory))

def attack_territory(territory_buttons, small_section, root, current_player):
    global territories2

    # Display a label asking the player to choose a territory to attack from
    choose_territory_label = ctk.CTkLabel(small_section, text="Choose a territory
to attack from")
    choose_territory_label.pack(pady=10)

```

```

def select_from_territory(territory):
    nonlocal choose_territory_label

    # Check if the territory is owned by the current player and has more than
1 troop
    if territory.owner == current_player and territory.armies > 1:
        choose_territory_label.pack_forget() # Remove the "Choose a
territory" Label

        # Display a label asking the player to choose a territory to attack
        choose_territory_label = ctk.CTkLabel(small_section, text="Choose a
territory to attack")
        choose_territory_label.pack(pady=10)

    def select_to_territory(to_territory):
        nonlocal choose_territory_label

        # Check if the territory to attack is not owned by the current
player
        if to_territory.owner != current_player:
            choose_territory_label.pack_forget() # Remove the "Choose a
territory" Label

            # Create a new window to prompt for the number of troops to
attack with
            attack_window = ctk.CTkToplevel(root)
            attack_window.title("Attack")
            attack_window.attributes("-topmost", True) # Make the window
appear above the root window

            # Create a label displaying the number of troops in the
"from" territory
            troops_label = ctk.CTkLabel(attack_window, text=f"Number of
troops in {territory.name}: {territory.armies}")
            troops_label.pack(pady=10)

            # Create an entry field for the number of troops to attack
with
            troops_entry = ctk.CTkEntry(attack_window)
            troops_entry.pack(pady=10)

            # Define a function to handle the attack
            def perform_attack():
                nonlocal attack_window

```

```

# Get the number of troops to attack with from the entry
field
troops_count = troops_entry.get()

# Validate the input
try:
    troops_count = int(troops_count)
    if 1 <= troops_count < territory.armies:
        # Subtract the troops from the "from" territory
        territory.armies -= troops_count

        # Update the territory button
        update_territory_buttons(territory_buttons,
territory)

        # Perform the attack
        attack_result = resolve_attack(territory,
to_territory, troops_count)

        # Update the territory buttons based on the
attack result

        if attack_result:
            to_territory.set_owner(current_player)
            to_territory.armies = troops_count
            update_territory_buttons(territory_buttons,
to_territory)

        else:
            territory.armies = 1

        # Close the attack window
        attack_window.destroy()

        # Switch to the next player's turn
        switch_player()
    else:
        # Display an error message if the input is
invalid
        ctk.CTkLabel(attack_window, text="Invalid input.
Please enter a number between 1 and one less than the total
troops.").pack(pady=10)
except ValueError:
    # Display an error message if the input is not a
number
    ctk.CTkLabel(attack_window, text="Invalid input.
Please enter a number.").pack(pady=10)

```

```

        # Create a button to confirm the attack
        confirm_button = ctk.CTkButton(attack_window, text="Attack",
command=perform_attack)
        confirm_button.pack(pady=10)

    else:
        # Display an error message if the territory to attack is
owned by the current player
        ctk.CTkLabel(small_section, text=f"{to_territory.name} is
owned by {current_player.name}").pack(pady=10)

        # Bind the select_to_territory function to each territory button
        for button in territory_buttons.values():
            button.bind("<Button-1>", lambda event,
territory=button.territory: select_to_territory(territory))

    else:
        # Display an error message if the territory is not owned by the
current player or has 1 troop
        if territory.owner != current_player:
            ctk.CTkLabel(small_section, text=f"{territory.name} is not owned
by {current_player.name}").pack(pady=10)
        else:
            ctk.CTkLabel(small_section, text=f"{territory.name} has only 1
troop").pack(pady=10)

        # Bind the select_from_territory function to each territory button
        for button in territory_buttons.values():
            button.bind("<Button-1>", lambda event, territory=button.territory:
select_from_territory(territory))

def resolve_attack(from_territory, to_territory, troops_count):
    # Roll dice for the attacker (player)
    attacker_dice = roll_dice(min(troops_count, 3))

    # Roll dice for the defender (AI)
    defender_dice = roll_dice(min(to_territory.armies, 2))

    # Determine the winner of the attack
    attacker_wins = compare_dice_rolls(attacker_dice, defender_dice)

    # Create a new window to display the dice rolls and the result
    result_window = ctk.CTkToplevel()
    result_window.title("Attack Result")

```

```

    # Display the attacker's dice rolls
    attacker_label = ctk.CTkLabel(result_window,
text=f"{from_territory.owner.name}'s Dice Rolls: {'', '}.join(map(str,
attacker_dice)))")
    attacker_label.pack(pady=5)

    # Display the defender's dice rolls
    defender_label = ctk.CTkLabel(result_window,
text=f"{to_territory.owner.name}'s Dice Rolls: {'', '}.join(map(str,
defender_dice)))")
    defender_label.pack(pady=5)

    # Display the result of the attack
    if attacker_wins:
        result_label = ctk.CTkLabel(result_window,
text=f"{from_territory.owner.name} wins the attack!")
    else:
        result_label = ctk.CTkLabel(result_window,
text=f"{to_territory.owner.name} wins the attack!")
    result_label.pack(pady=10)

    # Close the result window after a delay
    # result_window.after(3000, result_window.destroy)

    return attacker_wins

def roll_dice(num_dice):
    import random
    return sorted([random.randint(1, 6) for _ in range(num_dice)], reverse=True)

def compare_dice_rolls(attacker_rolls, defender_rolls):
    for attacker_roll, defender_roll in zip(attacker_rolls, defender_rolls):
        if attacker_roll > defender_roll:
            return True
        elif attacker_roll < defender_roll:
            return False
    return len(attacker_rolls) > len(defender_rolls)

def gameWindow(winner):
    global current_player_label
    root = ctk.CTk()
    root.title("RiskGame")
    root.geometry("1600x800")

```



```

frame = ctk.CTkFrame(master=root, width=1600, height=800)
frame.pack(pady=0, padx=0, fill="both", expand=True)

# Create the Larger section for the RISK map
large_section = ctk.CTkCanvas(master=frame, width=1200, height=800,
highlightthickness=0)
large_section.place(x=0, y=0)

# Load the background image
map_image = Image.open("map.png")

# Get the size of the large section
section_width = 1200
section_height = 800

# Calculate the aspect ratio of the image and the large section
image_width, image_height = map_image.size
image_aspect_ratio = image_width / image_height
section_aspect_ratio = section_width / section_height

# Determine the zoom factor based on the aspect ratios
zoom_factor = section_width / image_width

# Create the resized PhotoImage
resized_image = map_image.resize((int(image_width * zoom_factor),
int(image_height * zoom_factor)), Image.Resampling.LANCZOS)
map_photo = ImageTk.PhotoImage(resized_image)

# Display the resized image on the canvas
large_section.create_image(0, 0, anchor="nw", image=map_photo)
large_section.image = map_photo # Keep a reference to prevent garbage
collection

global current_player
if winner.lower() == player.name.lower():
    current_player = player
else:
    current_player = ai

territory_buttons = create_territory_buttons(large_section, current_player)

# Create the smaller section for the buttons
small_section = ctk.CTkFrame(master=frame, width=400, height=800)
small_section.place(x=1200, y=0)

```

```
# Create a Label to display the current player's turn
current_player_label = ctk.CTkLabel(small_section,
text=f"{current_player.name}'s Turn", width=380)
current_player_label.pack(pady=10, fill="x")

# Create buttons in the smaller section
attack_button = ctk.CTkButton(small_section, text="Attack", width=380,
command=lambda: attack_territory(territory_buttons, small_section, root,
current_player))
attack_button.pack(pady=10, fill="x")

place_troop_button = ctk.CTkButton(small_section, text="Place Troop",
width=380, command=lambda:
place_troops(territory_buttons, small_section, root, current_player))
place_troop_button.pack(pady=10, fill="x")

move_troops_button = ctk.CTkButton(small_section, text="Move Troops",
width=380, command=lambda: switch_player())
move_troops_button.pack(pady=10, fill="x")

root.mainloop()
```